

STACKED PRs

One giant diff, split into reviewable layers.

THE MANUAL WAY vs GITHUB'S NATIVE WAY

8 slides · what changed, and what it costs you

ONE PR. 1,720 LINES.

The reviewer holds the data layer, the API,
and the UI in their head at the same time.

40 files. One scroll.
No natural stopping point.

Review quality drops before review speed does.

LAYERED BRANCHES

main

- └ feat/catalog-data -> PR to main
 - └ feat/search-api -> PR to catalog-data
 - └ feat/chat -> PR to search-api
 - └ feat/ui-> PR to chat

Each branch targets the one below it.
Git has no idea they belong together.

A chain of branches, tracked by hand.

CHANGE LAYER 1, REDO THE STACK.

```
$ git rebase feat/catalog-data main  
$ git push --force-with-lease feat/catalog-data  
$ git rebase --onto ... feat/search-api  
$ git push --force-with-lease feat/search-api  
$ git rebase --onto ... feat/chat  
$ git push --force-with-lease feat/chat
```

4 layers = 4 rebases, 4 force pushes.

WHAT GITHUB ADDED

- stack map in the PR header
- cascading rebase: server-side or gh stack rebase
- branch protection + CI run against the stack base
- merge the top PR -> every layer below lands
- stack object exposed via API and webhooks

The platform now knows the layers are related.

MERGING

Merge the TOP -> whole stack lands, bottom-up

Merge MID-STACK -> layers below land;

 layers above auto-retarget

Squash / rebase / merge commit: all supported

Merge queue aware: PRs enter in order

One merge event, not four.

THE FINE PRINT

- × linear history required – rebase or no merge
- × same repository only (no cross-fork stacks)
- × not supported in GitHub Desktop
- × no auto-merge on stacked PRs (yet)
- × closing a middle PR blocks everything above
- × 3-5 layers is the practical ceiling

Powerful, but the discipline is yours to keep.

AI MADE WRITING CODE CHEAP.

Review is the bottleneck now.

Smaller layers are not a style choice.
They are how review keeps up.